JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Enum Class RoundingMode

java.lang.Object
    java.lang.Enum<RoundingMode>
        java.math.RoundingMode

All Implemented Interfaces:

Serializable, Comparable<RoundingMode>, Constable

```
public enum RoundingMode
extends Enum<RoundingMode>
```

Specifies a *rounding policy* for numerical operations capable of discarding precision. Each rounding mode indicates how the least significant returned digit of a rounded result is to be calculated. If fewer digits are returned than the digits needed to represent the exact numerical result, the discarded digits will be referred to as the *discarded fraction* regardless the digits' contribution to the value of the number. In other words, considered as a numerical value, the discarded fraction could have an absolute value greater than one.

More generally, a rounding policy defines a mapping from the real numbers to a subset of representable values. In the case of `BigDecimal`, the representable values are a function of the precision being used in the computation. Assuming the mathematical result is within the exponent range of `BigDecimal`, the mathematical result will be exactly representable in the result precision or will fall between two adjacent representable values. In the case of falling between two representable values, the rounding policy determines which of those two bracketing values is the result. For in-range real numbers, for a given set of representable values, a rounding policy maps a continuous segment of the real number line to a single representable value where the real number numerically equal to a representable value is mapped to that value.

Each rounding mode description includes a table listing how different two-digit decimal values would round to a one digit decimal value under the rounding mode
in question. The result column in the tables could be gotten by creating a `BigDecimal` number with the specified value, forming a `MathContext` object with the proper settings (`precision` set to `1`, and the `roundingMode` set to the rounding mode in question), and calling `round` on this number with the proper `MathContext`. A summary table showing the results of these rounding operations for all rounding modes appears below.

Summary of Rounding Operations Under Different Rounding Modes

Input Number	Result of rounding input to one digit with the given rounding mode							
	UP	DOWN	CEILING	FLOOR	HALF_UP	HALF_DOWN	HALF_EVEN	UNNECESSARY
5.5	6	5	6	5	6	5	6	

Input Number	Result of rounding input to one digit with the given rounding mode							
	UP	DOWN	CEILING	FLOOR	HALF_UP	HALF_DOWN	HALF_EVEN	UNNECESSARY
								throw ArithmeticException
2.5	3	2	3	2	3	2	2	throw ArithmeticException
1.6	2	1	2	1	2	2	2	throw ArithmeticException
1.1	2	1	2	1	1	1	1	throw ArithmeticException
1.0	1	1	1	1	1	1	1	1
-1.0	-1	-1	-1	-1	-1	-1	-1	-1
-1.1	-2	-1	-1	-2	-1	-1	-1	throw ArithmeticException
-1.6	-2	-1	-1	-2	-2	-2	-2	throw ArithmeticException
-2.5	-3	-2	-2	-3	-3	-2	-2	throw ArithmeticException
-5.5	-6	-5	-5	-6	-6	-5	-6	throw ArithmeticException

This enum is intended to replace the integer-based enumeration of rounding mode constants in `BigDecimal` (`BigDecimal.ROUND_UP`, `BigDecimal.ROUND_DOWN`, etc.).

API Note:

Five of the rounding modes declared in this class correspond to rounding-direction attributes defined in the *IEEE Standard for Floating-Point Arithmetic*. Where present, this correspondence will be noted in the documentation of the particular constant.

See *Java Language Specification*:

15.4 Floating-point Expressions

Since:

1.5

External Specifications

IEEE Standard for Floating-Point Arithmetic

See Also:

BigDecimal, MathContext

Nested Class Summary

Nested classes/interfaces declared in class Enum
Enum.EnumDesc<E>

Enum Constant Summary

Enum Constants

Enum Constant	Description
CEILING	Rounding mode to round towards positive infinity.
DOWN	Rounding mode to round towards zero.
FLOOR	Rounding mode to round towards negative infinity.
HALF_DOWN	Rounding mode to round towards "nearest neighbor" unless both neighbors are equidistant, in which case round down.
HALF_EVEN	Rounding mode to round towards the "nearest neighbor" unless both neighbors are equidistant, in which case, round towards the even neighbor.
HALF_UP	Rounding mode to round towards "nearest neighbor" unless both neighbors are equidistant, in which case round up.
UNNECESSARY	Rounding mode to assert that the requested operation has an exact result, hence no rounding is necessary.
UP	Rounding mode to round away from zero.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
static RoundingMode	**valueOf**(int rm)	Returns the RoundingMode object corresponding to a legacy integer rounding mode constant in BigDecimal.
static RoundingMode	**valueOf**(String name)	Returns the enum constant of this class with the specified name.
static RoundingMode[]	**values**()	Returns an array containing the constants of this enum class, in the order they are declared.

Methods declared in class Enum

clone, compareTo, describeConstable, equals, finalize, getDeclaringClass, hashCode, name, ordinal, toString, valueOf

Methods declared in class Object

getClass, notify, notifyAll, wait, wait, wait

Enum Constant Details

UP

```
public static final RoundingMode UP
```

Rounding mode to round away from zero. Always increments the digit prior to a non-zero discarded fraction. Note that this rounding mode never decreases the magnitude of the calculated value.

Example:

Rounding mode UP Examples

Input Number	Input rounded to one digit with `UP` rounding
5.5	6
2.5	3
1.6	2
1.1	2
1.0	1
-1.0	-1
-1.1	-2
-1.6	-2
-2.5	-3
-5.5	-6

DOWN

```
public static final RoundingMode DOWN
```

Rounding mode to round towards zero. Never increments the digit prior to a discarded fraction (i.e., truncates). Note that this rounding mode never increases the magnitude of the calculated value.

API Note:

This rounding mode is analogous to the rounding policy used for the `float` and `double` operators remainder and conversion to an integer value (JLS §15.4). This mode corresponds to the IEEE 754 rounding-direction attribute roundTowardZero.

Example:

Rounding mode DOWN Examples

Input Number	Input rounded to one digit with `DOWN` rounding
5.5	5
2.5	2
1.6	1
1.1	1
1.0	1
-1.0	-1
-1.1	-1
-1.6	-1
-2.5	-2
-5.5	-5

CEILING

`public static final RoundingMode CEILING`

Rounding mode to round towards positive infinity. If the result is positive, behaves as for `RoundingMode.UP`; if negative, behaves as for `RoundingMode.DOWN`. Note that this rounding mode never decreases the calculated value. This mode corresponds to the IEEE 754 rounding-direction attribute roundTowardPositive.

Example:

Rounding mode CEILING Examples

Input Number	Input rounded to one digit with `CEILING` rounding
5.5	6
2.5	3
1.6	2
1.1	2
1.0	1
-1.0	-1
-1.1	-1
-1.6	-1
-2.5	-2
-5.5	-5

FLOOR

`public static final RoundingMode FLOOR`

Rounding mode to round towards negative infinity. If the result is positive, behave as for `RoundingMode.DOWN`; if negative, behave as for `RoundingMode.UP`. Note that this rounding mode never increases the calculated value. This mode corresponds to the IEEE 754 rounding-direction attribute roundTowardNegative.

Example:

Rounding mode FLOOR Examples

Input Number	Input rounded to one digit with `FLOOR` rounding
5.5	5
2.5	2
1.6	1

Input Number	Input rounded to one digit with `FLOOR` rounding
1.1	1
1.0	1
-1.0	-1
-1.1	-2
-1.6	-2
-2.5	-3
-5.5	-6

HALF_UP

```
public static final RoundingMode HALF_UP
```

Rounding mode to round towards "nearest neighbor" unless both neighbors are equidistant, in which case round up. Behaves as for `RoundingMode.UP` if the discarded fraction is ≥ 0.5; otherwise, behaves as for `RoundingMode.DOWN`. Note that this is the rounding mode commonly taught at school. This mode corresponds to the IEEE 754 rounding-direction attribute roundTiesToAway.

Example:

Rounding mode HALF_UP Examples

Input Number	Input rounded to one digit with `HALF_UP` rounding
5.5	6
2.5	3
1.6	2
1.1	1
1.0	1
-1.0	-1
-1.1	-1
-1.6	-2
-2.5	-3
-5.5	-6

HALF_DOWN

```
public static final RoundingMode HALF_DOWN
```

Rounding mode to round towards "nearest neighbor" unless both neighbors are equidistant, in which case round down. Behaves as for `RoundingMode.UP` if the discarded fraction is > 0.5; otherwise, behaves as for `RoundingMode.DOWN`.

Example:

Rounding mode HALF_DOWN Examples

Input Number	Input rounded to one digit with `HALF_DOWN` rounding
5.5	5
2.5	2
1.6	2
1.1	1
1.0	1
-1.0	-1
-1.1	-1
-1.6	-2
-2.5	-2
-5.5	-5

HALF_EVEN

```
public static final RoundingMode HALF_EVEN
```

Rounding mode to round towards the "nearest neighbor" unless both neighbors are equidistant, in which case, round towards the even neighbor. Behaves as for `RoundingMode.HALF_UP` if the digit to the left of the discarded fraction is odd; behaves as for `RoundingMode.HALF_DOWN` if it's even.

API Note:

This is the rounding mode that statistically minimizes cumulative error when applied repeatedly over a sequence of calculations. It is sometimes known as "Banker's rounding," and is chiefly used in the USA. This rounding mode is analogous to the rounding policy used for most `float` and `double` arithmetic operators in Java (JLS §15.4). This mode corresponds to the IEEE 754 rounding-direction attribute roundTiesToEven.

Example:

Rounding mode HALF_EVEN Examples

Input Number	Input rounded to one digit with `HALF_EVEN` rounding
5.5	6
2.5	2
1.6	2
1.1	1
1.0	1

Input Number	Input rounded to one digit with `HALF_EVEN` rounding
-1.0	-1
-1.1	-1
-1.6	-2
-2.5	-2
-5.5	-6

UNNECESSARY

```
public static final RoundingMode UNNECESSARY
```

Rounding mode to assert that the requested operation has an exact result, hence no rounding is necessary. If this rounding mode is specified on an operation that yields an inexact result, an `ArithmeticException` is thrown.

Example:

Rounding mode UNNECESSARY Examples

Input Number	Input rounded to one digit with `UNNECESSARY` rounding
5.5	throw `ArithmeticException`
2.5	throw `ArithmeticException`
1.6	throw `ArithmeticException`
1.1	throw `ArithmeticException`
1.0	1
-1.0	-1
-1.1	throw `ArithmeticException`
-1.6	throw `ArithmeticException`
-2.5	throw `ArithmeticException`
-5.5	throw `ArithmeticException`

Method Details

values

```
public static RoundingMode[] values()
```

Returns an array containing the constants of this enum class, in the order they are declared.

Returns:

an array containing the constants of this enum class, in the order they are declared

valueOf

`public static RoundingMode valueOf(String name)`

Returns the enum constant of this class with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this class. (Extraneous whitespace characters are not permitted.)

Parameters:

`name` - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

`IllegalArgumentException` - if this enum class has no constant with the specified name

`NullPointerException` - if the argument is null

valueOf

`public static RoundingMode valueOf(int rm)`

Returns the `RoundingMode` object corresponding to a legacy integer rounding mode constant in `BigDecimal`.

Parameters:

`rm` - legacy integer rounding mode to convert

Returns:

`RoundingMode` corresponding to the given integer.

Throws:

`IllegalArgumentException` - integer is out of range

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.